40%	25%	20%	15%
Fetch works Phase 1	Image-driven agent Phase 1	User verdict Phase 2	Stretch goal

TWO PHASES

PHASE	GOAL
Phase 1 — Robotics build	A typed command goes in; a completed Fetch comes out without human help. Target: working by hour 14. Phase 2 cannot start before mentor sign-off.
Phase 2 — Human feedback	Record the robot’s point-of-view clip and submit it through Membrane’s human feedback API. Real users judge whether the robot did the task.

What teams must build.

Three processes work together: the game engine, a Python agent and the Claude API. The engine owns the scene, physics and camera rendering. The agent runs the tool-use loop and calls engine endpoints. Claude receives the command, latest camera frame, optional depth reading and recent history.

PROCESS	RESPONSIBILITY
Engine	Runs scene at 30 FPS or more; renders robot camera; hosts local HTTP server at localhost:8080; owns physics, collisions and state machine.
Agent	Python script holds the command, runs Claude tool-use loop, calls engine endpoints and writes logs.
Claude API	Receives command, latest frame, depth value and recent history. Returns exactly one tool call per step.

Required scene and task

• One flat room with four walls and no furniture.
• 4–6 primitive shapes on the floor (cubes, spheres, cylinders, cones) in different colours. At least two share a colour and at least two share a shape.
• One marked drop zone, such as a coloured square or flat platform.
• One wheeled robot with a forward-facing camera; a capsule with a camera is enough.
• One plain-language command per run. Judges may use reasoning commands such as “Bring the round red thing” or “Bring the tallest object.”

Robot behaviour and state machine

The robot finds the object from its camera, drives to it, picks it up, carries it to the drop zone and releases it. Pick-up is a snap on contact; grasping physics is not required. Collisions must stop the robot and be visible in the clip.

STATE	DESCRIPTION
Idle	Waits for command.
Locate	Claude identifies target in camera view.
Go to object	Robot moves until within pickup reach.
Pick up	Robot code attaches nearest object.
Carry	Robot moves while holding object toward zone.
Drop	Robot releases object in zone.
Done	Reports pass or fail.
Blocked	Collision or no movement for 2 seconds; back off/turn/retry. Fail after 3 retries.

Run outcomes: pass, fail_wrong_object, fail_not_reached or fail_collision. A run ends when the robot reaches the drop zone with an object or gives up after 3 retries.

The camera is the source of object information.

The engine knows everything about the scene. The agent and Claude must not receive object information from the engine. Object data stops at the engine boundary.

ENDPOINT	FUNCTION
GET /state	State name, robot x/y, heading, whether holding something and retry count. No object information.
GET /look	JPEG frame from robot camera. Claude's only view of objects.
GET /depth (optional)	Single-ray distance to nearest thing straight ahead. Does not identify the thing.
POST /move {distance}	Moves forward/backward; stops early on collision and returns collision status.
POST /turn {degrees}	Turns robot in place.
POST /pickup	Attaches nearest object if within reach. Returns success/failure only; never identifies object.
POST /drop	Releases held object and reports whether it landed in the zone.

CRITICAL — PROHIBITED OBJECT DATA
Do not send object names, IDs, colours, shapes, positions, distances to objects, object counts, segmentation masks, named-target distances or drop-zone coordinates to Claude. This applies to /state, system prompts, tool descriptions, filenames and all context. Everything Claude knows about objects comes from /look pixels, plus optional depth and collision flag.

Optional sensors

LIKELY APPROVABLE (MENTOR REVIEW)	ALWAYS REJECTED
Multi-ray 360° depth scan; side bump sensors; second/wider camera; downward camera for drop zone; wheel odometry; IMU.	Object IDs, names, colours, shapes, positions, counts; distance to named target; segmentation masks; drop-zone coordinates.

Extra sensors require mentor approval, normally within 15 minutes. Approved sensors are published to all teams within the hour; no private sensors.

Control loop

- One Claude call per step; expect 15–40 steps per run. Each image call takes 2–5 seconds.
- Freeze engine clock while Claude thinks; scene advances only during /move and /turn.
- Send /look as an image content block, not a text description.
- Take one /look after each /move or /turn. Never send two consecutive images without an intervening move or turn.
- Log every step as JSONL: step, timestamp, tool, args, result, frame_path. Save frames as PNG beside the log.

Technical contract & budget.

ITEM	SPECIFICATION
Units	Metres for distance; degrees for angles. Heading 0 = +x axis; positive clockwise from above.
/look	JPEG 640 × 480; 90° horizontal field of view. Camera at robot front, 0.5 m above floor, tilted 10° down.
/depth	Single ray from camera centre; float metres, max 10; null if nothing in range.
/move	Body {distance}; negative = reverse; speed 1 m/s.
/turn	Body {degrees}; positive = right; speed 90°/s.
/pickup	Reach 0.5 m and within ±30° of heading.
Collision	Stops move, backs off 0.1 m, returns collided=true and enters Blocked.
Log	One JSONL line per step with frame path; frames saved as PNG.

Claude API budget guidance

LIMIT	GUIDANCE
Target	$40 per team; mentor alert at $30. No hard block.
Images per step	One /look image before each decision.
Steps per run	25 soft target; 60 hard stop.
Estimated cost	About $0.009 per step; ~$0.25 at 25 steps; ~$0.55 at 60 steps.
Run capacity	Approximately 150 runs within $40, with development/rehearsal/hidden-test buffer.

Budget-saving guidance: test HTTP plumbing with Haiku or a dummy agent before using Sonnet; enable prompt caching; send only the latest image; keep earlier steps as one-line text; count runs. At 120 runs, stop development and reserve budget for rehearsal and hidden commands.

PHASE 1 COMPLETION CHECK

Type a command and reach Done with pass on at least 3 different commands. Logs must show Claude found each object from /look images. Nobody may touch keyboard or mouse during the run. Demo live to a mentor; sign-off unlocks Phase 2 API access.

How teams are scored.

The supplied Membrane problem statement assigns 100% across four components: Phase 1 is 65%, Phase 2 is 20%, and stretch goals are 15%.

COMPONENT	WEIGHT	HOW IT IS MEASURED
Phase 1: Fetch works	40%	Live demo on two judge commands. Pass/fail per command, plus how directly the robot went.
Phase 1: Image-driven agent	25%	Judges read tool-call and image logs. Did Claude find the object from pixels, recover from collision and reason about the command?
Phase 2: User verdict	20%	Pulse 1 yes-rate and Pulse 2 “nothing wrong” share, averaged across the three submitted clips.
Stretch goal	15%	Any one listed stretch goal shown working in the demo.

Detailed judging interpretation

40 POINTS — PHASE 1: FETCH WORKS
Judges run two commands. Each command is assessed pass/fail: did the robot fetch the requested object and deliver it to the drop zone? Judges also consider directness of travel; unnecessary wandering reduces performance.

25 POINTS — PHASE 1: IMAGE-DRIVEN AGENT
Judges inspect tool-call logs and image logs. Evidence should show Claude identifying the target from camera pixels, reasoning across views and recovering from collisions. Any object-data leakage from the engine scores 0 on both Phase 1 components.

20 POINTS — PHASE 2: USER VERDICT
Each of the three clips receives at least 30 user responses. Pulse 1 asks if the correct object reached the drop zone (yes/no). Pulse 2 asks what went wrong: hit something, wrong object, did not reach drop zone, too slow or nothing wrong. The yes-rate and “nothing wrong” share are averaged across the three clips.

15 POINTS — STRETCH GOAL
Any one working stretch goal counts: improve behaviour using pulse results and show before/after; add furniture or walls blocking a straight path; have Claude plan the full path with move/turn only (no go_to); or handle multi-view reasoning such as finding the cube farthest from the green square.

Tie-breaker: Phase 1 demo score.

What the crowd sees and judges.

Phase 2 starts only after mentor sign-off of Phase 1. Teams then receive a human feedback API key and one-page API guide.

POV clip requirements

REQUIREMENT	SPECIFICATION
Format	MP4, 720p or higher; no audio needed.
Duration	10–20 seconds.
Recording	Start when state leaves Idle; stop when it reaches Done.
Command overlay	Show typed command as text for first 2 seconds.
Prohibited on screen	No team names, logos or code visible.

Human-feedback questions

PULSE 1 — BINARY	PULSE 2 — LABEL
“Did the robot bring the correct object to the drop zone?” — Yes or no.	“What went wrong?” Choose: hit something; wrong object; did not reach the drop zone; too slow; nothing wrong.

Hidden commands and deadlines

WHEN	REQUIREMENT
First 20 hours	Develop with your own commands.
Hour 18	Judges publish three unseen commands. Run each command once.
By hour 20	Upload all three clips. Only these three clips go to the pulses.
Within 45 minutes	Receive result sheet per clip: Pulse 1 yes-rate and Pulse 2 label split.

PHASE 2 ZERO-SCORE CONDITIONS
No clips uploaded by hour 20 means 0 for Phase 2. Clips with team names/logos/code visible or duration over 20 seconds are not sent to pulses and score 0 for Phase 2.

24-hour execution plan.

Phase 1 fills the first 14 hours. Phase 2 has two hard gates: hidden commands at hour 18 and clip upload at hour 20.

MILESTONE	TARGET
Build scene	Create flat room, primitive shapes, drop zone and wheeled robot camera.
Expose robot API	Implement local HTTP endpoints for state, look, movement, turning, pickup and drop.
Connect Claude	Build Python agent and tool-use loop. Test HTTP plumbing before spending vision API calls.
Phase 1 sign-off	Target hour 14. Pass at least 3 different commands, show image-based identification in logs, no human input during run; get mentor sign-off.
Phase 2 setup	Receive feedback API key/guide after sign-off and record POV footage.
Hidden tests	At hour 18, run each of the three unseen commands once.
Clip upload	Upload all three compliant clips by hour 20.

Stretch goals — any one counts

- Use pulse result sheet to change robot behaviour and show before/after clips.
- Add furniture or walls that block the straight path.
- Have Claude plan the full path using move and turn only, without go_to.
- Handle multi-view reasoning, such as “Bring the cube that is farthest from the green square.”

Submission checklist

SUBMIT ALL FOUR

- ■ Public repository link with engine project and Claude agent code.
- ■ Tool-call log for each of the three hidden-command runs.
- ■ One paragraph explaining how Claude finds the object from the image.
- ■ Demo slot booked for a live run on one judge command.

What organizers provide

One Claude API key per team (Sonnet 5.5, $40 target, mentor alert at $30, vision enabled); fixed pulse questions; human feedback API key and one-page guide after Phase 1 sign-off; operator support if an API call fails; reference scene description and sample command list; mentors for engine and Claude tool-use questions.

SOURCE NOTE

Based on the supplied “Membrane Robot Hackathon — Problem Statement” (Nov 2, 2026). Scoring weights, constraints, API details, milestones, feedback questions and submission requirements are summarized from that source. No additional scoring weights have been invented.